

# WSTunnel over WireGuard Implementation Report

## 1. Objective

The organization firewall blocked native **WireGuard UDP (51820)** traffic, preventing on-premises virtual machines from connecting to the AWS VPN server. To overcome this limitation, **WSTunnel** was implemented to encapsulate WireGuard UDP packets inside **WebSocket over HTTPS (TCP 443)**, allowing VPN traffic to traverse restrictive firewalls while maintaining secure encrypted communication.

---

## 2. Problem Statement

### Existing VPN Architecture

Client

|

WireGuard (UDP 51820)

|

Internet

|

AWS VPN Server

### Issue

- Organization firewall blocked:
  - UDP/51820
  - ICMP
- VPN tunnel could not establish.
- No administrative control over the organization's firewall.

### Requirement

Encapsulate VPN traffic inside HTTPS traffic that is typically permitted by enterprise firewalls.

---

## 3. Solution Architecture

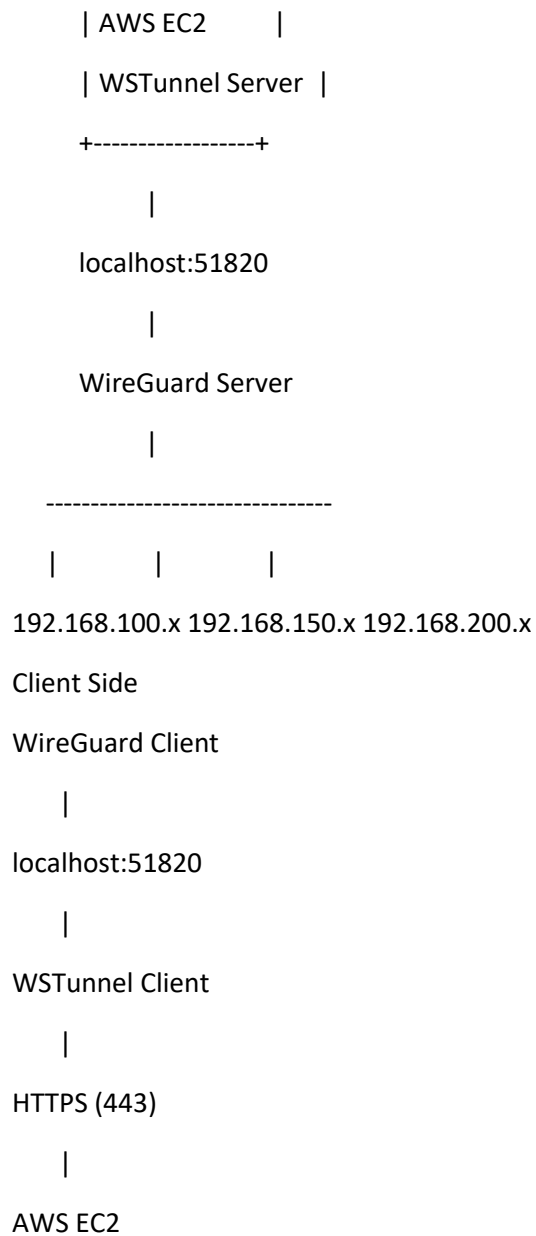
Internet

|

HTTPS (TCP 443)

|

+-----+



#### 4. Infrastructure

##### AWS EC2 VPN Server

##### Component Configuration

|            |               |
|------------|---------------|
| OS         | Ubuntu        |
| Public IP  | 13.205.137.92 |
| WSTunnel   | Server Mode   |
| WireGuard  | Server        |
| HTTPS Port | TCP 443       |

**Component Configuration**

VPN Port     UDP 51820

Firewall     nftables

---

**Client**

Implemented on:

- Ubuntu Linux
- Windows Server

Components

- WireGuard Client
- WSTunnel Client

---

**5. VPN Addressing**

VPN Server

192.168.100.1/24

192.168.150.1/24

192.168.200.1/24

Configured Clients

| Client       | VPN IP          |
|--------------|-----------------|
| Web Server   | 192.168.100.10  |
| Web Server-2 | 192.168.100.50  |
| Honeypot     | 192.168.100.100 |
| Service VM   | 192.168.100.200 |
| MISP         | 192.168.150.10  |
| SOC Core     | 192.168.150.20  |
| IDS          | 192.168.150.30  |
| AD-DS        | 192.168.150.40  |

---

**6. Server Configuration**

## WireGuard

Server interface:

wg0

Listening:

UDP 51820

Addresses

192.168.100.1/24

192.168.150.1/24

192.168.200.1/24

Each VPN client was configured as a WireGuard peer using its public key and corresponding /32 AllowedIPs entry.

---

## WSTunnel Server

Installed as a systemd service.

Configuration

wstunnel server \

--restrict-to localhost:51820 \

wss://0.0.0.0:443

Purpose

- Listen on HTTPS port 443
- Accept WebSocket connections
- Forward decrypted UDP packets to localhost WireGuard server

---

## 7. Client Configuration

### WireGuard

Endpoint

127.0.0.1:51820

Reason

WireGuard communicates locally with WSTunnel instead of directly with AWS.

---

### WSTunnel Client

wstunnel client \

-L udp://51820:localhost:51820?timeout\_sec=0 \

wss://13.205.137.92:443

Function

WireGuard

|

localhost:51820

|

WSTunnel

|

HTTPS 443

|

AWS

---

## 8. Routing Configuration

Initially

AllowedIPs = 0.0.0.0/0

This caused:

- Entire system traffic routed into VPN
- WSTunnel HTTPS packets attempted to traverse the VPN they were responsible for establishing
- Routing loop
- Connection timeouts

Solution

Added a static route before bringing up WireGuard:

```
ip route add 13.205.137.92/32 via <physical_gateway> dev <physical_interface>
```

Result

Traffic destined for the AWS public IP bypassed the VPN and reached WSTunnel directly, while all other traffic continued through the VPN.

---

## 9. Firewall Configuration

**AWS Security Group**

Allowed

| Protocol | Port |
|----------|------|
|----------|------|

|     |    |
|-----|----|
| SSH | 22 |
|-----|----|

|       |     |
|-------|-----|
| HTTPS | 443 |
|-------|-----|

WireGuard UDP 51820

|      |    |
|------|----|
| HTTP | 80 |
|------|----|

---

## nftables

Configured to permit:

- SSH
- ICMP
- WireGuard UDP
- WSTunnel HTTPS
- VPN forwarding
- NAT masquerading

HTTPS rule was added after discovering inbound TCP 443 traffic was being dropped.

---

## 10. Windows Client Implementation

### Components

- WireGuard Windows
- WSTunnel
- NSSM (service manager)

### Initial Issue

WSTunnel failed to start with:

VCRUNTIME140.dll missing

### Resolution

Installed Microsoft Visual C++ Redistributable (2015–2022 x64), after which WSTunnel executed successfully.

---

### NSSM Service

Created to launch WSTunnel automatically at startup.

Service executed:

wstunnel.exe client \

-L udp://51820:localhost:51820?timeout\_sec=0 \

wss://13.205.137.92:443

---

## 11. Troubleshooting Summary

| Issue                           | Root Cause                                                | Resolution                                          |
|---------------------------------|-----------------------------------------------------------|-----------------------------------------------------|
| WSTunnel TCP timeout            | TCP 443 blocked by nftables                               | Allowed HTTPS traffic                               |
| curl to public IP timed out     | Firewall blocking HTTPS                                   | Updated nftables                                    |
| Invalid request from curl       | HTTP request sent instead of WebSocket upgrade            | Expected behavior; server confirmed reachable       |
| WSTunnel connection timeout     | HTTPS traffic routed into VPN due to AllowedIPs=0.0.0.0/0 | Added static route for AWS public IP                |
| Server rejected destination     | --restrict-to configuration mismatch                      | Corrected server restriction to localhost:51820     |
| Windows WSTunnel service failed | Missing Visual C++ runtime                                | Installed VC++ Redistributable                      |
| NSSM service misconfiguration   | Service created without proper arguments                  | Recreated service with correct client command       |
| WireGuard handshake absent      | WSTunnel transport unavailable                            | Resolved after HTTPS connectivity and routing fixes |

---

## 12. Validation Performed

Server

- Verified WSTunnel service status.
- Confirmed WireGuard interface operational.
- Checked listening ports (443, 51820).
- Validated nftables rules.
- Tested HTTPS locally and via public IP.

Client

- Confirmed WSTunnel service started.

- Verified HTTPS connectivity to AWS.
- Confirmed WireGuard handshake.
- Verified VPN IP assignment.
- Tested inter-VPN communication.

---

### 13. Final Communication Flow

Application

|

WireGuard Client

|

127.0.0.1:51820 (UDP)

|

WSTunnel Client

|

WebSocket over TLS

TCP 443

|

Internet

|

AWS EC2

|

WSTunnel Server

|

localhost:51820 (UDP)

|

WireGuard Server

|

VPN Network

|

192.168.100.0/24

192.168.150.0/24



## 14. Key Findings

- WSTunnel effectively bypassed organizational restrictions on UDP by tunneling WireGuard traffic through HTTPS (TCP 443).
- A full-tunnel WireGuard configuration (AllowedIPs = 0.0.0.0/0) introduced a routing loop that prevented WSTunnel from reaching the AWS endpoint; a static host route resolved this issue.
- The WSTunnel server's --restrict-to option must explicitly permit the intended local UDP destination (localhost:51820) to allow forwarding to WireGuard.
- curl responses of "Invalid request" confirmed the HTTPS listener was reachable, but they did not validate WSTunnel functionality because WSTunnel requires a WebSocket upgrade rather than a standard HTTP request.
- On Windows, the Microsoft Visual C++ Runtime is a mandatory dependency for wstunnel.exe.
- Hosting WSTunnel as a persistent service (systemd on Linux, NSSM on Windows) ensured automatic startup and reliable operation.
- The final design successfully enabled Linux and Windows clients to join the private VPN over TCP 443 without exposing WireGuard UDP to the internet, providing a secure and firewall-friendly transport layer.

## WSTunnel over WireGuard Implementation Report

### Phase 1 – Preparation, Server Configuration & WSTunnel Server Deployment

---

#### 1. Objective

The organization's firewall allowed only **TCP 80, TCP 443 and SSH**, while **UDP 51820** used by WireGuard was blocked.

Instead of replacing WireGuard, the VPN architecture was modified by placing **WSTunnel** between the client and WireGuard server.

Final communication:

WireGuard

|

UDP localhost:51820

|

WSTunnel

|

HTTPS (TCP 443)

|

Internet

|

AWS EC2

|

WSTunnel

|

UDP localhost:51820

|

WireGuard Server

This implementation required **no modification to the existing VPN topology**, only the transport mechanism changed.

---

## 2. Existing Infrastructure

### AWS VPN Server

| Item                | Value                                                |
|---------------------|------------------------------------------------------|
| OS                  | Ubuntu 22.04 LTS                                     |
| Public IP           | 13.205.137.92                                        |
| WireGuard Interface | wg0                                                  |
| WireGuard Port      | UDP 51820                                            |
| VPN Networks        | 192.168.100.0/24, 192.168.150.0/24, 192.168.200.0/24 |

WireGuard status before implementation:

```
sudo wg show
```

Output confirmed

interface: wg0

listening port: 51820

along with all configured peers.

---

## 3. Verify Existing WireGuard

Before implementing WSTunnel the existing VPN was verified.

Command

```
sudo wg show
```

Checks performed

- WireGuard interface active
- Listening on UDP 51820
- Existing peers connected
- VPN routing operational

No configuration changes were made to WireGuard at this stage.

---

#### **4. Verify HTTPS Port Availability**

WSTunnel would occupy TCP 443.

Verify nothing is listening:

```
sudo ss -tlnp | grep :443
```

Expected

(no output)

Port 443 available.

---

#### **5. DNS Issue Encountered**

While downloading WSTunnel:

```
wget https://github.com/erebe/wstunnel/releases/latest/download/...
```

Error

Temporary failure in name resolution

Verification

```
ping 8.8.8.8
```

Worked successfully.

Therefore

- Internet connectivity existed.
- DNS resolution failed.

Checked

```
cat /etc/resolv.conf
```

Resolved by temporarily bringing WireGuard down.

```
sudo wg-quick down wg0
```

DNS immediately started functioning.

Verification

```
dig www.google.com
```

Successful.

Finding

WireGuard DNS configuration was temporarily interfering with external name resolution during installation.

---

## 6. Install WSTunnel

Initially attempted

```
wget  
https://github.com/erebe/wstunnel/releases/latest/download/wstunnel_10.5.5_linux_amd64.tar.gz
```

Result

404 Not Found

Reason

GitHub redirected latest release to

v10.6.2

while filename remained

10.5.5

Latest release verified using

```
wget -qO- https://api.github.com/repos/erebe/wstunnel/releases/latest \  
| grep browser_download_url
```

Correct package

wstunnel\_10.6.2\_linux\_amd64.tar.gz

Installation

```
cd /tmp
```

```
wget  
https://github.com/erebe/wstunnel/releases/download/v10.6.2/wstunnel_10.6.2_linux_amd64.tar.  
gz
```

```
tar -xzf wstunnel_10.6.2_linux_amd64.tar.gz
```

```
sudo mv wstunnel /usr/local/bin/
```

```
sudo chmod +x /usr/local/bin/wstunnel
```

Verification

```
wstunnel --version
```

Output

```
wstunnel-cli 10.6.2
```

---

## 7. Study WSTunnel Options

Reviewed server help

```
wstunnel server --help
```

Important options identified

| Option                              | Purpose                         |
|-------------------------------------|---------------------------------|
| --restrict-to                       | Restrict destination forwarding |
| --tls-certificate                   | Custom TLS certificate          |
| --tls-private-key                   | Private key                     |
| --restrict-http-upgrade-path-prefix | Optional authentication path    |
| --dns-resolver                      | DNS configuration               |

Decision

Use embedded self-signed certificate for initial deployment.

---

## 8. Start WSTunnel Server

Command

```
sudo wstunnel server \  
--restrict-to localhost:51820 \  
wss://0.0.0.0:443
```

Purpose

- Listen on HTTPS port 443
  - Accept WebSocket clients
  - Forward UDP traffic only to localhost WireGuard server
- 

## 9. Verify WSTunnel Server

Check listening socket

```
sudo ss -tlnp | grep :443
```

Expected

LISTEN

0.0.0.0:443

Verify locally

```
curl -vk https://127.0.0.1:443
```

Response

HTTP/2 400

Invalid request

Finding

This **did not indicate an error**.

Reason

WSTunnel expects

HTTP Upgrade: websocket

A normal HTTP GET request is intentionally rejected.

Therefore

Invalid request

confirmed

- HTTPS reachable
  - TLS functioning
  - WSTunnel server running
- 

## 10. External HTTPS Verification

Initially

`curl -vk https://13.205.137.92:443`

Timed out.

Investigation revealed

AWS Security Group already permitted

TCP 443

Root cause

Host firewall.

---

## **11. nftables Investigation**

Existing rules

`sudo nft list ruleset`

Input policy

policy drop

Allowed

SSH

ICMP

UDP 51820

Missing

TCP 443

Therefore HTTPS packets never reached WSTunnel.

---

## **12. Firewall Modification**

Added HTTPS rule.

Example

`sudo nft add rule inet filter input tcp dport 443 accept`

Verification

`curl -vk https://13.205.137.92:443`

Now received

HTTP/2 400

Invalid request

Finding

The server was now reachable from the Internet.

---

### 13. Convert WSTunnel into a Service

Created

/etc/systemd/system/wstunnel.service

Configuration

[Unit]

Description=WSTunnel Server

After=network-online.target

Wants=network-online.target

[Service]

Type=simple

ExecStart=/usr/local/bin/wstunnel server \

--restrict-to localhost:51820 \

wss://0.0.0.0:443

Restart=always

RestartSec=5

User=root

[Install]

WantedBy=multi-user.target

Reload

sudo systemctl daemon-reload

Enable

sudo systemctl enable wstunnel

Start

sudo systemctl start wstunnel

Status



sudo systemctl status wstunnel

Expected

Active: active (running)

---

## 14. Validation

Verified

sudo journalctl -u wstunnel

Confirmed

Starting WSTunnel Server

Listening on 0.0.0.0:443

Server ready.

---

## Phase 1 Summary

### Completed

- Verified existing WireGuard VPN
- Identified firewall restrictions
- Resolved DNS issue during installation
- Installed WSTunnel v10.6.2
- Studied server configuration options
- Deployed WSTunnel server
- Opened TCP 443 in nftables
- Validated HTTPS listener
- Created persistent systemd service
- Confirmed WSTunnel server operational

### Important Findings

- WSTunnel does **not** replace WireGuard; it encapsulates WireGuard UDP traffic inside WebSocket over HTTPS.
- A response of "**HTTP/2 400 Invalid request**" from curl is expected because WSTunnel requires a WebSocket upgrade request, not a standard HTTP GET.
- The AWS Security Group already allowed TCP 443; the actual blocker was the local nftables firewall.

- Running WSTunnel as a systemd service ensures automatic startup after reboot.

**Phase 2** will cover **Linux client implementation**, including WSTunnel client setup, WireGuard integration, routing changes (AllowedIPs issue), troubleshooting, and achieving a successful VPN handshake over HTTPS.

**WSTunnel over WireGuard Implementation Report**

**Phase 2 – Linux Client Configuration, WireGuard Integration & Routing**

---

**1. Objective**

Configure the Linux VPN client to:

- Connect to the AWS WSTunnel server over HTTPS (TCP 443)
  - Forward local WireGuard traffic through WSTunnel
  - Preserve the existing WireGuard VPN architecture
  - Bypass organizational firewall restrictions
- 

**2. Client Environment**

| Component               | Value            |
|-------------------------|------------------|
| OS                      | Ubuntu 22.04 LTS |
| Host                    | VM-SERVICES-01   |
| VPN Address             | 192.168.100.10   |
| AWS Server              | 13.205.137.92    |
| WireGuard Interface wg0 |                  |

---

**3. Install WSTunnel**

Determine system architecture

uname -m

cat /etc/os-release

Output

x86\_64

Ubuntu 22.04 LTS

Download

```
cd /tmp
```

```
wget
```

```
https://github.com/erebe/wstunnel/releases/download/v10.6.2/wstunnel\_10.6.2\_linux\_amd64.tar.gz
```

```
tar -xzf wstunnel_10.6.2_linux_amd64.tar.gz
```

```
sudo mv wstunnel /usr/local/bin/
```

```
sudo chmod +x /usr/local/bin/wstunnel
```

```
Verify
```

```
wstunnel --version
```

```
Output
```

```
wstunnel-cli 10.6.2
```

---

#### 4. Configure WireGuard

Original Endpoint

Endpoint = 13.205.137.92:51820

Modified

Endpoint = 127.0.0.1:51820

Reason

WireGuard no longer communicates directly with AWS.

Instead

WireGuard

|

127.0.0.1:51820

|

WSTunnel

|

HTTPS

|

AWS

---

## 5. Start WSTunnel Client

Command

```
wstunnel client \  
-L udp://51820:127.0.0.1:51820?timeout_sec=0 \  
wss://13.205.137.92:443
```

Meaning

| Option          | Description                 |
|-----------------|-----------------------------|
| client          | Client mode                 |
| -L              | Local UDP forward           |
| udp://51820     | Listen locally on UDP 51820 |
| 127.0.0.1:51820 | Forward to local WireGuard  |
| timeout_sec=0   | Never timeout UDP           |
| wss://          | Secure WebSocket            |
| :443            | HTTPS transport             |

---

## 6. Verify WSTunnel

Socket

```
sudo ss -ltnp | grep 51820
```

Expected

```
127.0.0.1:51820
```

Process

```
wstunnel
```

---

## 7. Configure WSTunnel as Service

Create

```
/etc/systemd/system/wstunnel.service
```

Contents

```
[Unit]
```

Description=WSTunnel Client

After=network-online.target

Wants=network-online.target

[Service]

Type=simple

ExecStart=/usr/local/bin/wstunnel client \

-L udp://51820:127.0.0.1:51820?timeout\_sec=0 \

wss://13.205.137.92:443

Restart=always

RestartSec=5

User=root

[Install]

WantedBy=multi-user.target

Reload

sudo systemctl daemon-reload

Enable

sudo systemctl enable wstunnel

Start

sudo systemctl start wstunnel

Status

sudo systemctl status wstunnel

---

## 8. Start WireGuard

Bring interface up

sudo wg-quick up wg0

Verify

sudo wg show

Initially

Transfer:

0 B received

Packets only transmitted

Meaning

WireGuard attempted communication, but no response was received.

---

## 9. Initial Troubleshooting

Logs

```
sudo journalctl -u wstunnel -f
```

Observed

Opening TCP connection

Cannot connect

Timed out

Meaning

HTTPS transport was failing.

---

## 10. Verify HTTPS Connectivity

Command

```
nc -vz 13.205.137.92 443
```

Initially

Timeout

Later

Connection succeeded

Finding

Transport layer eventually became reachable after firewall corrections on the server.

---

## 11. Major Routing Problem

Client configuration contained

AllowedIPs = 0.0.0.0/0

Purpose

Full tunnel VPN.

Unexpected effect

All traffic

|

WireGuard

|

WSTunnel

|

AWS

Problem

WSTunnel itself also attempted to use the VPN.

Result

WSTunnel

↓

VPN

↓

WSTunnel

↓

VPN

↓

Infinite routing loop

Symptoms

- HTTPS timeout

- No handshake
  - Increasing transmitted bytes
  - Zero received bytes
- 

## 12. Investigation

Routes

ip route

Observed

default via physical gateway

After WireGuard

Entire default route redirected into VPN.

Confirmed by

nc -vz 13.205.137.92 443

Succeeded only after bringing WireGuard down.

Finding

WireGuard itself caused WSTunnel connectivity failure.

---

## 13. Solution

Create static route before VPN starts.

Command

```
sudo ip route add 13.205.137.92/32 \
```

```
via <physical_gateway> \
```

```
dev <physical_interface>
```

Example

```
sudo ip route add 13.205.137.92/32 \
```

```
via 192.168.74.254 \
```

```
dev ens33
```

Verify

ip route

Expected

13.205.137.92 via 192.168.74.254



Now

AWS Server



Physical NIC

NOT



WireGuard

---

#### **14. Integrate with WireGuard**

Added to wg0.conf

PostUp = ip route add 13.205.137.92/32 via 192.168.74.254 dev ens33

PostDown = ip route del 13.205.137.92/32 via 192.168.74.254 dev ens33

Reason

Automatically preserve the route whenever WireGuard starts.

---

#### **15. Restart VPN**

sudo wg-quick down wg0

sudo wg-quick up wg0

Check routes

ip route

Expected

default

13.205.137.92

VPN routes

---

## 16. Verify WireGuard

`sudo wg show`

Now

Latest handshake

Transfer RX increasing

Transfer TX increasing

Handshake successful.

---

## 17. Verify Communication

Ping

`ping 192.168.100.1`

Result

Successful.

VPN traffic now flowed

WireGuard

↓

localhost

↓

WSTunnel

↓

HTTPS



AWS



WireGuard



VPN

---

## 18. Final Client Architecture

Application



WireGuard



127.0.0.1:51820



WSTunnel Client



HTTPS (443)



Internet



AWS EC2



WSTunnel Server



127.0.0.1:51820

|

WireGuard Server

|

VPN Networks

---

## 19. Validation Commands

### WSTunnel

```
sudo systemctl status wstunnel
```

```
sudo journalctl -u wstunnel
```

```
sudo ss -ltnp | grep 51820
```

---

### WireGuard

```
sudo wg show
```

```
ip route
```

---

### Connectivity

```
nc -vz 13.205.137.92 443
```

```
ping 192.168.100.1
```

---

## 20. Key Findings

- WSTunnel transparently encapsulated WireGuard UDP traffic into HTTPS (TCP 443), allowing VPN operation through restrictive enterprise firewalls.
- The most significant issue was the interaction between a full-tunnel WireGuard configuration (AllowedIPs = 0.0.0.0/0) and WSTunnel: without an explicit host route, WSTunnel attempted to reach the AWS endpoint through the VPN it was responsible for establishing, creating a routing loop.
- Adding a static host route for the AWS public IP through the physical gateway resolved the loop while retaining full-tunnel behavior for all other traffic.
- Persisting the host route via PostUp and PostDown in wg0.conf ensured the configuration survived interface restarts.
- Once the route was corrected, WSTunnel connected successfully, WireGuard established a handshake, and VPN communication between the client and AWS server operated normally over HTTPS.

